Introduction to Haskell Programming

Recursive data types

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Recursive data types

- Just like we have recursive functions, we can have recursive data types

Recursive data types

- Just like we have recursive functions, we can have recursive data types
- Recursive datatype `t` – some components are of the same type `t`

Recursive data types

- Just like we have recursive functions, we can have recursive data types
- Recursive datatype `t` – some components are of the same type `t`
- Some constructors of a recursive data type `t` have `t` among their input types, as well as the return type

Example: Nat

- Simplest recursive data type

```
data Nat = Zero | Succ Nat

Zero :: Nat
Succ :: Nat → Nat
```

Functions on `Nat`

- Check for zero:

```
isZero :: Nat → Bool
isZero Zero = True
isZero _    = False
```

Functions on `Nat`

- Check for zero:

```
isZero :: Nat → Bool
isZero Zero = True
isZero _    = False
```

- Predecessor (or previous number):

```
prev :: Nat → Nat
prev Zero     = Zero
prev (Succ n) = n
```

Functions on `Nat`

- Addition:

```
plus :: Nat → Nat → Nat
plus m Zero     = m
plus m (Succ n) = Succ (plus m n)
```

Functions on Nat

- Addition:

```
plus :: Nat → Nat → Nat
plus m Zero     = m
plus m (Succ n) = Succ (plus m n)
```

- Multiplication:

```
mult :: Nat → Nat → Nat
mult m Zero     = Zero
mult m (Succ n) = plus m (mult m n)
```

Showing Nat

- A custom **show** for Nat:

```
data Nat = Zero | Succ Nat
instance Show Nat where
    show = show . turnToInt
turnToInt :: Nat → Int
turnToInt Zero = 0
turnToInt (Succ n) = turnToInt n + 1
```

Showing Nat

- A custom **show** for Nat:

```
data Nat = Zero | Succ Nat
instance Show Nat where
    show = show . turnToInt
turnToInt :: Nat → Int
turnToInt Zero = 0
turnToInt (Succ n) = turnToInt n + 1
```

- In **show = show .** turnToInt

Showing Nat

- A custom **show** for Nat:

```
data Nat = Zero | Succ Nat
instance Show Nat where
    show = show . turnToInt
turnToInt :: Nat → Int
turnToInt Zero = 0
turnToInt (Succ n) = turnToInt n + 1
```

- In **show** = **show** . turnToInt
  - The left **show** has type Nat → **String**

Showing `Nat`

- A custom **`show`** for `Nat`:

```
data Nat = Zero | Succ Nat
instance Show Nat where
    show = show . turnToInt
turnToInt :: Nat → Int
turnToInt Zero = 0
turnToInt (Succ n) = turnToInt n + 1
```

- In **`show = show . turnToInt`**
  - The left **`show`** has type `Nat → String`
  - The right **`show`** has type `Int → String`

Example: `List`

- Recursive data types can also be polymorphic

```
List a = Nil | Cons a (List a)
```

Example: `List`

- Recursive data types can also be polymorphic

```
List a = Nil | Cons a (List a)
```

- This is the built-in type `[a]`

Example: `List`

- Recursive data types can also be polymorphic

```
List a = Nil | Cons a (List a)
```

- This is the built-in type `[a]`
- Functions are defined as usual on pattern matching:

```
head :: List a → a
head (Cons x _) = x
```

Example: `List`

- Recursive data types can also be polymorphic

```
List a = Nil | Cons a (List a)
```

- This is the built-in type `[a]`
- Functions are defined as usual on pattern matching:

```
head :: List a → a
head (Cons x _) = x
```

- Exception on `head Nil`

List

- **List** and **head**

```
List a = Nil | Cons a (List a)
head :: List a → a
head (Cons x _) = x
```

List

- **List** and **head**

```
List a = Nil | Cons a (List a)
head :: List a → a
head (Cons x _) = x
```

- Exception on **head** Nil

List

- **List** and **head**

```
List a = Nil | Cons a (List a)
head :: List a → a
head (Cons x _) = x
```

- Exception on **head** Nil
- Can fix it with custom **head**

```
head :: List a → Maybe a
head Nil        = Nothing
head (Cons x _) = Just x
```

Folds on `List`

- We can make `List` an instance of **`Foldable`**

```
instance Foldable List where
    foldr :: (a → b → b) → b → List a → b
    foldr f v Nil           = v
    foldr f v (Cons x xs)   = f x (foldr f v xs)
```

Folds on `List`

- We can make `List` an instance of **`Foldable`**

```
instance Foldable List where
    foldr :: (a → b → b) → b → List a → b
    foldr f v Nil           = v
    foldr f v (Cons x xs)   = f x (foldr f v xs)
```

- Now we can define functions using any of the **`fold`** variants

```
import Data.Foldable ( foldl' )
listSum :: Num a ⇒ List a → a
listSum     = foldl' (+) 0
```

Arithmetic expressions

- Consider arithmetic expressions involving integers and $+, -, \times, /$

Arithmetic expressions

- Consider arithmetic expressions involving integers and $+, -, \times, /$
- Operators have **precedence** and **associativity**

Arithmetic expressions

- Consider arithmetic expressions involving integers and $+$, $-$, $\times$, $/$
- Operators have **precedence** and **associativity**
  - $\times$ and $/$ have greater precedence than $+$ and $-$

Arithmetic expressions

- Consider arithmetic expressions involving integers and $+$, $-$, $\times$, $/$
- Operators have **precedence** and **associativity**
  - $\times$ and $/$ have greater precedence than $+$ and $-$
  - $3 + 5 \times 2$ evaluates to 13

Arithmetic expressions

- Consider arithmetic expressions involving integers and $+$, $-$, $\times$, $/$
- Operators have **precedence** and **associativity**
  - $\times$ and $/$ have greater precedence than $+$ and $-$
  - $3 + 5 \times 2$ evaluates to 13
  - $6 - 4 - 2$ evaluates to 0

Arithmetic expressions

- Consider arithmetic expressions involving integers and $+$, $-$, $\times$, $/$
- Operators have **precedence** and **associativity**
  - $\times$ and $/$ have greater precedence than $+$ and $-$
  - $3 + 5 \times 2$ evaluates to 13
  - $6 - 4 - 2$ evaluates to 0
  - $22/8/2$ evaluates to 1

Arithmetic expressions

- Consider arithmetic expressions involving integers and $+, -, \times, /$
- Operators have **precedence** and **associativity**
  - $\times$ and $/$ have greater precedence than $+$ and $-$
  - $3 + 5 \times 2$ evaluates to 13
  - $6 - 4 - 2$ evaluates to 0
  - $22/8/2$ evaluates to 1
  - Parentheses to indicate precedence – $22/(8/2) = 5$ (**integer division**)

Arithmetic expressions in Haskell

- Natural recursive definition

Arithmetic expressions in Haskell

- Natural recursive definition
  - Any integer is an expression

Arithmetic expressions in Haskell

- Natural recursive definition
  - Any integer is an expression
  - An operator applied on two expressions is an expression

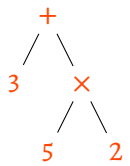
Arithmetic expressions in Haskell

- Natural recursive definition
  - Any integer is an expression
  - An operator applied on two expressions is an expression
- An expression has a **tree structure**, with integer values at the **leaves**, and operators at the **nodes**

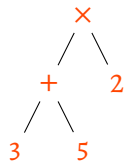
```
data Exp = Val Int | Add Exp Exp | Sub Exp Exp | Mul Exp Exp | Div Exp Exp
    deriving Eq
```

Arithmetic expressions in Haskell

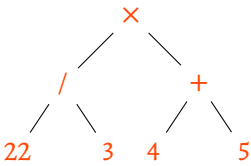
$exp_1 = 3 + 5 \times 2$



$exp_2 = (3 + 5) \times 2$



$exp_3 = 22/3 \times (4 + 5)$




```
exp1, exp2, exp3 :: Exp
exp1 = Add (Val 3) (Mul (Val 5) (Val 2))
exp2 = Mul (Add (Val 3) (Val 5)) (Val 2)
exp3 = Mul (Div (Val 22) (Val 3)) (Add (Val 4) (Val 5))
```

Displaying expressions

```
instance Show Exp where
    show (Val n) = show n
    show (Add expl expr) = "(" ++ show expl ++ " + " ++ show expr ++ ")"
    show (Sub expl expr) = "(" ++ show expl ++ " - " ++ show expr ++ ")"
    show (Mul expl expr) = "(" ++ show expl ++ " * " ++ show expr ++ ")"
    show (Div expl expr) = "(" ++ show expl ++ " / " ++ show expr ++ ")"
```

Evaluating expressions

```
eval :: Exp → Int
eval (Val n)        = n
eval (Add e1 e2)    = eval e1 + eval e2
eval (Sub e1 e2)    = eval e1 - eval e2
eval (Mul e1 e2)    = eval e1 * eval e2
eval (Div e1 e2)    = eval e1 `div` eval e2
```

Parsing expressions

- Writing values of type `Exp` is cumbersome

```
Add (Val 3) (Mul (Val 5) (Val 2))
```

Parsing expressions

- Writing values of type `Exp` is cumbersome

```
Add (Val 3) (Mul (Val 5) (Val 2))
```

- Much easier to just write `3 + 5 * 2`

Parsing expressions

- Writing values of type `Exp` is cumbersome

```
Add (Val 3) (Mul (Val 5) (Val 2))
```

- Much easier to just write `3 + 5 * 2`
- But we need to convert this to a value of type `Exp` before evaluating

Parsing expressions

- Writing values of type `Exp` is cumbersome

```
Add (Val 3) (Mul (Val 5) (Val 2))
```

- Much easier to just write `3 + 5 * 2`
- But we need to convert this to a value of type `Exp` before evaluating
- Interpreting strings as proper data values is called **parsing**

Parsing expressions

- Writing values of type `Exp` is cumbersome

```
Add (Val 3) (Mul (Val 5) (Val 2))
```

- Much easier to just write `3 + 5 * 2`
- But we need to convert this to a value of type `Exp` before evaluating
- Interpreting strings as proper data values is called **parsing**
- Non-trivial problem

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments
- In **postfix expressions**, operator is written after the arguments

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments
- In **postfix expressions**, operator is written after the arguments
- Postfix expressions do not require parentheses to indicate precedence or associativity

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments
- In **postfix expressions**, operator is written after the arguments
- Postfix expressions do not require parentheses to indicate precedence or associativity
- More amenable to parsing

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments
- In **postfix expressions**, operator is written after the arguments
- Postfix expressions do not require parentheses to indicate precedence or associativity
- More amenable to parsing
- Examples

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments
- In **postfix expressions**, operator is written after the arguments
- Postfix expressions do not require parentheses to indicate precedence or associativity
- More amenable to parsing
- Examples
  - $exp_1 = 3\ 5\ 2\ \times\ +$

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments
- In **postfix expressions**, operator is written after the arguments
- Postfix expressions do not require parentheses to indicate precedence or associativity
- More amenable to parsing
- Examples
  - $exp_1 = 3\,5\,2\,\times\,+$
  - $exp_2 = 3\,5\,+\,2\,\times$

Postfix expressions

- Operators in arithmetic expressions are **infix** – operator is in between two arguments
- In **postfix expressions**, operator is written after the arguments
- Postfix expressions do not require parentheses to indicate precedence or associativity
- More amenable to parsing
- Examples
  - $exp_1 = 3\,5\,2\,\times\,+$
  - $exp_2 = 3\,5\,+\,2\,\times$
  - $exp_3 = 2\,2\,3\,/\,4\,5\,+\,\times$

Postfix expressions in Haskell

- We represent these expressions by a list of **tokens**

Postfix expressions in Haskell

- We represent these expressions by a list of **tokens**
- Each token is either an integer or an operator

```
type Token = Either Int (Exp → Exp → Exp)
strToTokList :: String → [Token]
strToTokList str = map tokenize (words str) where
    tokenize :: String → Token
    tokenize "+" = Right Add
    tokenize "-" = Right Sub
    tokenize "*" = Right Mul
    tokenize "/" = Right Div
    tokenize str = Left (read str :: Int)
```

Building an expression

- We can build an expression from a valid list of tokens `toks`

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`
- Read the list `toks` from left to right

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`
- Read the list `toks` from left to right
- If the next token is **`Left`** `n`, add `Val n` to the front of `es`

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`
- Read the list `toks` from left to right
- If the next token is **`Left`** `n`, add `Val n` to the front of `es`
- If the next token is **`Right`** `op`

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`
- Read the list `toks` from left to right
- If the next token is `Left n`, add `Val n` to the front of `es`
- If the next token is `Right op`
  - extract first two expressions from `es`

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`
- Read the list `toks` from left to right
- If the next token is `Left n`, add `Val n` to the front of `es`
- If the next token is `Right op`
  - extract first two expressions from `es`
  - build a new expression with those two as subexpressions

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`
- Read the list `toks` from left to right
- If the next token is `Left n`, add `Val n` to the front of `es`
- If the next token is `Right op`
  - extract first two expressions from `es`
  - build a new expression with those two as subexpressions
  - add the new expression to the front of `es`

Building an expression

- We can build an expression from a valid list of tokens `toks`
- Start with an empty list of expressions `es`
- Read the list `toks` from left to right
- If the next token is `Left n`, add `Val n` to the front of `es`
- If the next token is `Right op`
  - extract first two expressions from `es`
  - build a new expression with those two as subexpressions
  - add the new expression to the front of `es`
- Finally return the head of `es`

Building an expression

```
tokListToExp :: [Token] → Exp
tokListToExp toks                    = e where
    (e:_)                            = foldl step [] toks
    step :: [Exp] → Token → [Exp]
    step es             (Left n)     = Val n: es
    step (exp2:exp1:es) (Right op)   = op exp1 exp2: es

strToExp :: String → Exp
strToExp    = tokListToExp . strToTokList

evalExpStr :: String → Int
evalExpStr  = eval . strToExp
```